Tiered Cache-HNSW: Using Hierarchical Caching System in HNSW

Rei Masuda
Graduate School of Science for Creative Emergence
Kagawa University
Takamatsu, Japan
s24g357@kagawa-u.ac.jp

Kazuma Iwamoto
Graduate School of Science for Creative Emergence
Kagawa University
Takamatsu, Japan
s24g351@kagawa-u.ac.jp

Kazuaki Ando
Faculty of Engineering and Design
Kagawa University
Takamatsu, Japan
ando.kazuaki@kagawa-u.ac.jp

Hitoshi Kamei
Integrated Center for Informatics
Kagawa University
Takamatsu, Japan
kamei.hitoshi@kagawa-u.ac.jp

2025 1st International Conference on Consumer Technology (ICCT-Pacific) | 979-8-3315-0412-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCT-PACIFIC63901.2025.11012786

***Abstract*— With the progress in generative AI technology, systems such as RAG (Retrieval-Augmented Generation) are increasingly used. In these systems, graph-based Approximate Nearest Neighbor Search (ANNS) is commonly used for background information retrieval. However, as the size of the dataset increases, the search speed becomes significantly slower. In this paper, we propose Tiered Cache-HNSW (TC-HNSW) that caches pre-categorized datasets in layers. TC-HNSW prioritizes caching frequently accessed nodes into faster storage based on the number of accesses and their category, allowing faster searches. This paper describes TC-HNSW system design, caching strategy, and the efficiency of categorization-based caching. Our preliminary evaluation shows that categorization-based caching improves performance, achieving search speeds four times faster than traditional methods when search topics are specified.**

***Keywords*— RAG, Hierarchical Caching, Categorization**

I. Introduction

Recently, Large Language Models have improved in response performance. The use of Retrieval-Augmented Generation (RAG) [1], which combines finding information from a database and text generation, is growing in organizations. RAG often uses vector databases based on HNSW (Hierarchical Navigable Small World) [2] graphs for searches. HNSW is currently the most widely used graph-based search algorithm for vector search. Meanwhile, when using graph-based search methods, memory usage increases significantly as the system scales. If the system offloads the node data in HNSW to disk, the search speed may slow down. This problem is important in typical RAG systems running on local servers. To mitigate slowdown, caching systems are often implemented in such cases.

However, simple page caching is not efficient for graph-based systems like HNSW, as it can only cache accessed vectors. Therefore, the challenge is how to efficiently use memory and disk space in medium-scale environments while keeping memory usage optimal and maintaining speed. We propose a system with a caching mechanism designed for specific use cases. By analyzing trends in frequently searched topics and dynamically applying hierarchical caching, we aim to achieve both faster search speeds and efficient cache.

II. Back Ground

A. Large-Scale Nearest Neighbor Search

Nearest Neighbor Search (NNS) is very important in AI fields like information retrieval, pattern recognition, and machine learning. However, when datasets grow larger, the "curse of dimensionality" becomes a problem. NNS is 100% accurate because it processes all data points. Meanwhile, this causes a high computational cost, especially for large-scale datasets. As a result, NNS cannot meet the needs for search speed and cost efficiency. Many studies into Approximate Nearest Neighbor Search (ANNS) have been proposed, which makes the search faster while allowing for some inaccuracy [3].

In RAG, vector databases using ANNS are used to search for embedded documents. ANNS finds approximate nearest neighbors in datasets containing high-dimensional vectors. There are four main types of ANNS algorithms based on the index structure [3]. Recently, graph-based methods have become popular because they can get accurate results with fewer data points without searching the whole dataset.

One well-known graph-based ANNS method is Hierarchical Navigable Small World (HNSW) [2]. HNSW uses a hierarchical graph with multiple layers to perform efficient nearest neighbor search. Each layer has fewer nodes at the top and more nodes at lower levels. The upper layers have long-distance links (connections to far-away points), and the lower layers have short-distance links (connections to nearby points). In the first part of the search, the upper layers are used to quickly find rough neighbors, and in the second part, the lower layers are used to find more accurate neighbors. This makes it possible to search large datasets quickly and accurately.

B. Conventional Graph-Based Search Methods

Graph-based search methods like HNSW need to load the whole graph structure into memory to search fast, which makes it hard to handle large datasets. Therefore, some methods have been proposed that focus on storing data on disk. DiskANN [4] is one of the popular implementations, which builds a flat and disk-efficient graph index using the Vamana algorithm. Also, there are methods to make high-dimensional vector data smaller, with Product Quantization (PQ) [5] being a well-known technique. These methods allow for fast searches with less memory while keeping search accuracy.

As RAG becomes more popular, the need for graph search algorithms running on local systems in organizations grows, and improving efficiency on medium-scale systems becomes more important. For example, if a company uses RAG in its departments, security-conscious organizations likely build the system locally to protect sensitive information that should not to be shared externally. In sales departments, searches focus on

customers and transactions, while in technical departments, searches focus on about product design and development. However, since the data source is shared across the company, different departments often access different parts of the data. Conventional methods [4] assumed the use of large, billion-scale graphs for general purposes. Thus, it is not suitable for medium-scale systems with such access patterns. Therefore, we propose Tiered Cache-HNSW (TC-HNSW), a backend search system that combines topic-based node categorization and a hierarchical caching system, offering more efficient and cost-effective services for specific use cases.

Topic-based node categorization is grouping graph nodes by the topic or theme of the data. For example, sales nodes can be grouped as "customers" or "transactions." Managing groups in tiered storage systems, data is stored in appropriate tier based on access frequency by category. This allows faster searches using limited memory space.

In this paper, we find out how well existing vector databases perform to think about the need for caching and the specific strategies for TC-HNSW. We also evaluate the effect of categorization for search speed and the nodes become biased.

III. TC-HNSW System Design

A. System Overview

The TC-HNSW overview is shown in Fig. 2. This system adds the methods of topic-based node categorization and tier management to the vector search database that uses the HNSW algorithm. It implements a hierarchical caching mechanism that stores data in layers as needed. In this system, all data is always in the lowest tier, and it is copied (cached) to higher tiers as needed. The system has the following three modules.

1) HNSW Graph Search Module: This module holds the HNSW structure and performs the nearest neighbor search based on user queries. During the search, this module collects accessed nodes and the search path, and sends the data to the Access Trend Analysis Module.

2) Access Trend Analysis Module: This part looks at the recent searches using the collected data and decides the tier for data to be cashed. The access frequency of nodes is measured by counting the number of accesses within a given period. These nodes are ranked in descending order based on their access frequency, and the system determines high-frequency nodes. The tier can be changed for individual nodes or for whole categories. Frequently accessed data is moved to a higher tier and infrequently accessed data to a lower one.

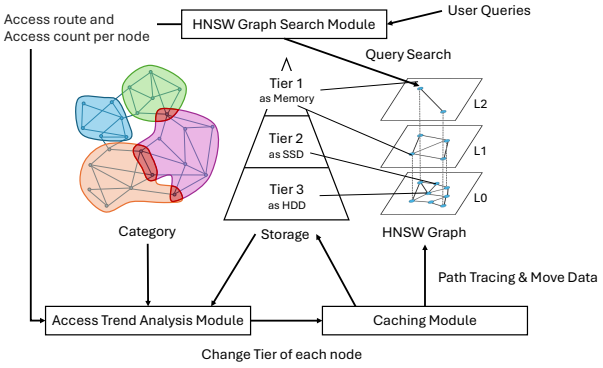

Fig. 2. System configuration diagram

3) Caching Module: This part performs extra graph searches and moves data among tiers when needed. Since caching can make a high system load, this process runs when the system load is low. The system can also pause this process when user queries are received.

B. Hierarchical Cache Design

TC-HNSW caches nodes based on the idea that queries about the same topic will access nearby vector data. It also assumes that HNSW nodes are categorized beforehand. The cache process follows four steps.

1) Caching of Accessed Nodes: Nodes retrieved from disk have their tier staged immediately. Since the graph structure itself is primarily stored in memory, the system caches only the node data. Unless the memory size is insufficient or the tier is changed, the node data is kept in memory. This is similar to normal page caching, which is based on the temporal locality.

2) Caching of Nodes in the Same Category: This runs at the lowest layer (Level 0, L0) of HNSW. For the final top-K nearest neighbor nodes found in a query, the system caches other nodes of the same category. This is based on the spatial locality.

3) Caching of Search Results Toward Upper Layers: While step 2 works at L0, in the higher layers, searches may pass through nodes of other categories, because the presence and connectivity of nodes in the upper layers affect the selection of entry nodes. Therefore, the system caches the node data along the path from the target category to the entry node in the highest layer, allowing it to cache nodes that might be searched later.

4) Tier Adjustment: The caching from the earlier steps runs for single queries. In this step, the system analyzes the trends from recent queries and stages the tier for nodes belonging to frequently accessed categories, while lowering the tier for nodes in categories that are accessed less often. The graph structure of the nodes remains unchanged, but the system updates the placement of node data and adjusts their reference paths. If a node has a high degree, it has a higher chance of being accessed, so its tier is more likely to stay the same. This is expected to keep the cache size from growing too large while keeping a high cache hit rate.

IV. Evaluation Experiment

A. Experimental Setup

For the evaluation, we used Qdrant [6] as the vector database. The dataset was ccnews [7]. Since this dataset has about 600 million articles, we only used the 2022 news data for our tests. Each article was split into its title and body, and we used M3-Embedding [8] to create embeddings for the experiments. The body of each article was treated as a node in the vector database, and the title was used as the query.

The server specifications used in the experiments are shown in Table I. We conducted two experiments:

1) Confirm search performance changes with different amounts of data.

TABLE I. SPECIFICATIONS OF EXPERIMENTAL SERVER

	Qdrant Server	Query Server
CPU	Ryzen7 5825U	Ryzen7 3700X
Memory	DDR4-3200 64GB	DDR4-3200 128GB
Platform	LXC, Docker Container	LXC, Docker Container
Storage	NVMe SSD 1TB	NVMe SSD 1TB
Network	1Gbps LAN	
Qdrant	Single node v1.9.5	Single node v1.9.5
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
Kernel	Linux 6.8	Linux 6.8

We confirmed search speed and system resource usage when we changed the amount of vector data stored in memmap storage. The search speed was measured by the official benchmarking system [9]. We changed the dataset size from one million to six million (1 M to 6 M) and measured search times with different "ef" (size of the candidate node set in search. A larger ef improves search accuracy but increases computation time) parameter values (64, 128, 256, 512). The base dataset had the top 120 categories from the 2022 news data (10 M entries). We made datasets of 1 M to 6 M by randomly sampling. We chose this setup because actual categorized datasets likely included different categories. Noisy data, such as empty categories or those labeled as "News," were removed. The queries were randomly selected from either the whole dataset or only from the Sports category. Each query set had 5,000 queries. We used the "sar" command to collect system load data every second and compared disk utilization (%util), which represents the percentage of time the disk is busy with I/O operations, and CPU usage (%sys + %usr) for 1M and 6M random queries.

2) Confirm categorization changes node access patterns.

We looked at the effects of the query topics for node access patterns in the HNSW graph. We used four categories—Sports, Politics, Business, and Entertainment—and ensured the dataset size ratios were balanced to avoid bias in the results for each category. The category information included in the ccnews was used to decide whether queries were on similar or different topics. Randomly selected queries represented different topics, while queries from a single category represented similar topics. In this paper, we used the Sports category as a representative example. Each query set had 5,000 queries, and "ef" was set to 64. We modified Qdrant to record how often each node was accessed during searches and checked the access patterns in the HNSW layers.

B. Experimental Results

Fig. 3 shows the system load from experiment 1 in section IV-A. The left graph shows %util, and the right graph shows %usr + %sys. Disk usage had room to spare at 1 M, and the CPU was used at around 70 %. Meanwhile, disk utilization reached nearly 100% at 6 M, and CPU usage dropped.

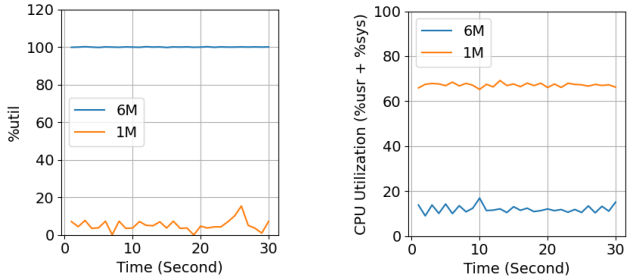

Fig. 3. %util and CPU utilization (%usr+%sys) during search with ef512.

Fig. 4 and Fig. 5 show search speed for each dataset size. In Fig. 4, random queries keep the response time within tens of milliseconds up to 2M, and the time becomes longer after 3M. For a 6M dataset, search response time reached 1.4 seconds per query with "ef" 512. In Fig. 5, the Sports queries showed a similar drop after 3M, but the decrease was only about 25% of that seen with random queries.

Fig. 6 shows a histogram of node access frequency in L0, with the x-axis representing access frequency (in bins of 10) and the y-axis representing the number of nodes. Table II shows the maximum node accesses and the difference between random and Sports queries across the HNSW layers as a percentage. Fig. 6 shows that Sports queries accessed fewer nodes compared to random queries, and certain nodes are accessed frequently. Table II shows a 20% difference in the access count of the most

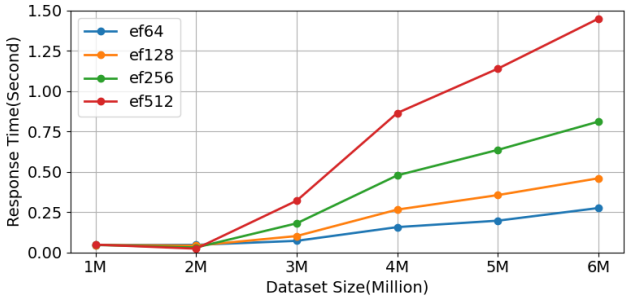

Fig. 4. Changes in response time for each "ef" as the dataset size increases (random queries)

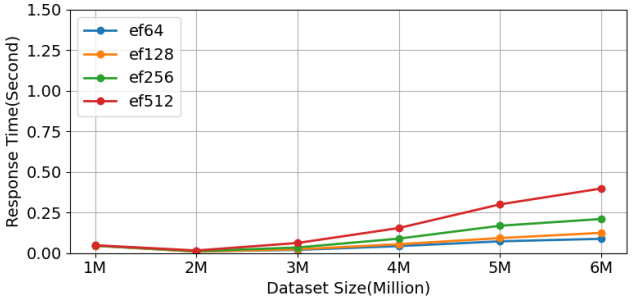

Fig. 5. Changes in response time for each "ef" as the dataset size increases (queries from the sports category)

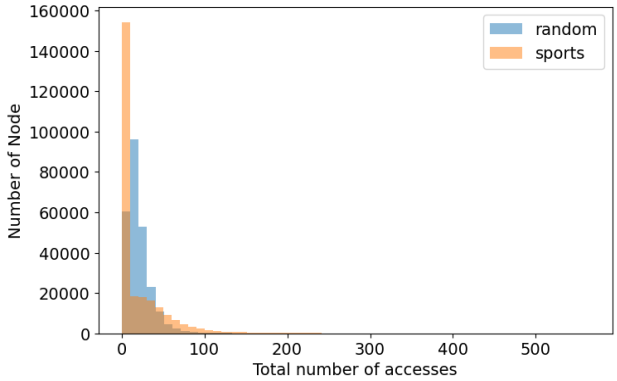

Fig. 6. Histogram comparing the number of nodes within ranges of access counts, divided into groups of 10, for random and sports category

TABLE II. MAXIMUM NUMBER OF ACCESSES PER NODE AT EACH LEVEL

Level	Max number of accesses per node		
	Random	***Sports***	***Sports / Random***
L0	246	564	229 %
L1	1116	1781	159 %
L2	2478	4962	200 %
L3	8576	15025	175 %
L4	16758	23163	137 %

frequently accessed nodes. The same trend is shown in from L1 to L4, though the difference becomes smaller.

C. Discussion

The results of the experiment 1 show that using memmap storage of HNSW leads to a sharp drop in performance when the dataset size goes beyond a certain point. From the evaluation, performance dropped after 2M, because disk access became a bottleneck. The search time for Sports queries is shorter compared to random queries. The reason is that Sports-related nodes were accessed more often; thus page cache worked better. This shows that categorizing datasets beforehand improves search speed. By using TC-HNSW, we expect to reduce the frequency of disk access for datasets larger than 3M, dynamically adjusting data placement based on recent access trends to enable fast and flexible searches.

The results of the experiment 2 show that categorization creates a clear bias in node access compared to random queries. When query topics are similar over a short period, TC-HNSW maintains fast search performance by limiting which nodes use higher-tier storage. Also, the greatest bias is seen at L0 compared to L1+. Therefore, TC-HNSW can optimize caching efficiency the most at L0.

These experiments show that TC-HNSW is effective for assumed use cases like department-level RAG, where the topics being searched may focus on certain areas for short periods. TC-HNSW focuses on datasets where user topics gather in specific areas, like in RAG, and it uses pre-set information to categorize the data. This approach allows for better long-term cache predictions, which is hard with just the graph structure. TC-HNSW adjusts data placement more precisely than similar methods while keeping a hierarchical structure and moves data during low-load times. TC-HNSW improves processing speed, maintains search quality and reduces memory usage.

V. Related Work

Graph Reordering [9] is a method to improve the cache hit rate of HNSW. This method does not change the graph structure but makes the search more efficient by changing node placement in memory. It tests six reordering methods including the "Gorder" algorithm which places nearby nodes in the graph close to each other, and "Degree Sorting" which arranges nodes by how many connections they have. The "Gorder" algorithm reduces CPU L3 cache misses by 40% and shortens query time by 20–30%.

HM-ANN [10] uses different types of memory to speed up searches and reduce memory usage for large datasets by changing the method of storing data for each HNSW layer. HM-ANN combines fast and small memory (like DRAM) with slower and larger memory (like NVRAM) to store large data. It enables fast and accurate searches. It also uses techniques like prefetching caches in L0 based on the search in L1+ and parallel searches. Compared to conventional HNSW or NSG [11] that use memmap storage and page caching, it is 2 to 5.8 times faster.

Previous studies mainly focused on the structure of the graph itself. TC-HNSW focuses on particular use cases where data might be categorized, such as department-level RAG. The experiments showed that TC-HNSW has the potential to improve caching efficiency more than traditional methods.

VI. Conclusion

In this paper, we proposed the caching system for vector databases in RAG. The proposed system, TC-HNSW, changes caching priorities based on node conditions to efficiently store vector data in memory. The proposed system pre-categorizes each vector and prioritizes caching vectors by the same category as the query. It assumes that there is some bias in the topics being searched. The evaluation results suggest that pre-categorizing vectors and a caching strategy by using categories is a useful method. As future work, we plan to implement the proposed caching system, evaluate it, and investigate the cost of pre-categorizing.

References

[1] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, arXiv:2312.10997. [Online]. Available: https://arxiv.org/abs/2312.10997

[2] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, April 2020.

[3] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," 2021, arXiv:2101.12631. [Online]. Available: https://arxiv.org/abs/2101.12631

[4] J. Subramanya, S. Suhas, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node," Advances in Neural Information Processing Systems, vol. 32, 2019.

[5] H. Jégou, M. Douze and C. Schmid, "Product Quantization for Nearest Neighbor Search," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 1, pp. 117–128, Jan. 2011.

[6] Qdrant, "What is Qdrant?," Accessed: Sep. 25, 2024. [Online]. Available: https://qdrant.tech/documentation/overview/

[7] Stanford-oval, "ccnews", Accessed: Aug. 19, 2024. Hugging Face Datasets, [Online]. Available: https://huggingface.co/datasets/stanford-oval/ccnews

[8] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," 2024, arXiv: 2402.03216. [Online]. Available: https://arxiv.org/abs/2402.03216

[9] Qdrant, "vector-db-benchmark," 2024, gitHub repository. [Online]. Available: https://github.com/qdrant/vector-db-benchmark

[10] B. Coleman, S. Segarra, A. J. Smola, and A. Shrivastava, "Graph Reordering for Cache-Efficient Near Neighbor Search," Advances in Neural Information Processing Systems, vol. 35, pp. 38488–38500, 2022.

[11] J. Ren, M. Zhang, and D. Li, "HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory," Advances in Neural Information Processing Systems, vol. 33, pp. 10672–10684, 2020.

[12] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph," 2017, arXiv:1707.00143. [Online]. Available: https://arxiv.org/abs/1707.00143